Taller de Lógica: Sincronización con Señales (IPC)

Malak Sanchez
Monitorias Sistemas Operativos

15 de octubre de 2026

Introducción

Este taller se enfoca en la coordinación asíncrona de procesos mediante el uso de señales POSIX (`SIGUSR1`, `SIGUSR2`). El objetivo es controlar el orden de ejecución de una jerarquía de procesos para que sigan una secuencia.

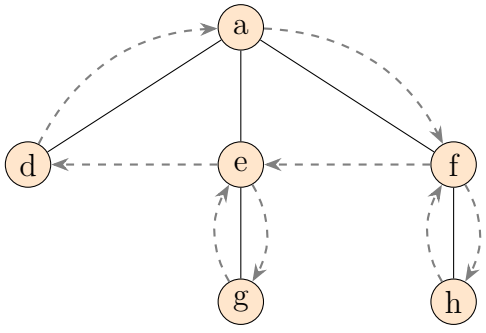
Bloque 1: De Diagramas a Código

Desarrolle el código necesario para replicar las siguientes estructuras y secuencias de señales. Todos los procesos deben imprimir su letra correspondiente al recibir o enviar el control.

Ejercicio 1

Replique la siguiente jerarquía. El flujo debe iniciar en la raíz, visitar primero la rama central, luego la derecha y finalmente la izquierda; cada retorno debe confirmarse con una señal distinta.

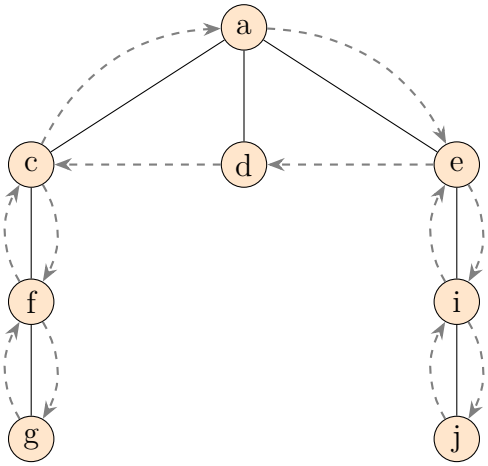
Secuencia sugerida: `a-e-g-e-f-h-f-d-a`



Ejercicio 2

Diseñe un programa que emule un recorrido alternado: complete la rama izquierda, active el nodo central y luego complete la rama derecha. El proceso raíz debe registrar cada confirmación.

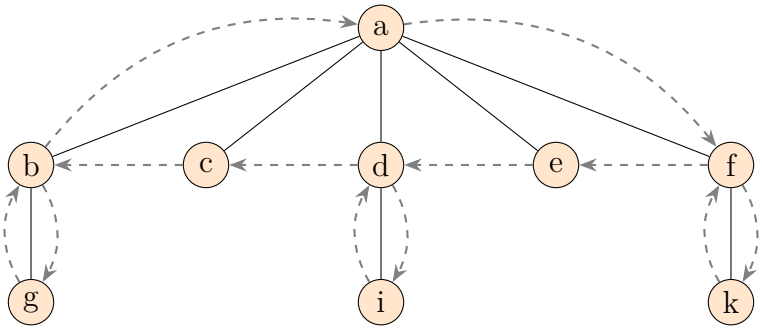
Secuencia sugerida: `a-c-f-g-f-c-d-e-i-j-i-e-a`



Ejercicio 3

Implemente la sincronización para una jerarquía combinada. Cada hoja debe devolver el control a la raíz antes de que esta habilite la siguiente rama; los hermanos no pueden señalizarse directamente.

Secuencia sugerida: `a-b-g-b-a-c-a-d-i-d-a-e-a-f-k-f-a`



Bloque 2: Análisis de Código

Determine el orden de impresión y explique qué señal libera cada espera. Considere que el padre tiene el control inicial.

Ejercicio 4

Analice el siguiente código y determine la salida exacta por consola. Asuma que `fork()` tiene éxito, que las señales no se pierden y que el padre inicia la ejecución.

```c
void handler(int sig) { /* Despierta el proceso */ }

int main(){
    signal(SIGUSR1, handler);
    pid_t p1, p2;
    p1 = fork();
    if(p1 == 0){
        pause();
        printf("Primero\n");
        kill(getppid(), SIGUSR1);
        exit(0);
    }
    p2 = fork();
    if(p2 == 0){
        pause();
        printf("Segundo\n");
        kill(getppid(), SIGUSR1);
        exit(0);
    }
    kill(p1, SIGUSR1);
    pause();
    kill(p2, SIGUSR1);
    pause();
    printf("Coordinador\n");
    return 0;
}
```